

Incremental Data Loading with a Watermark

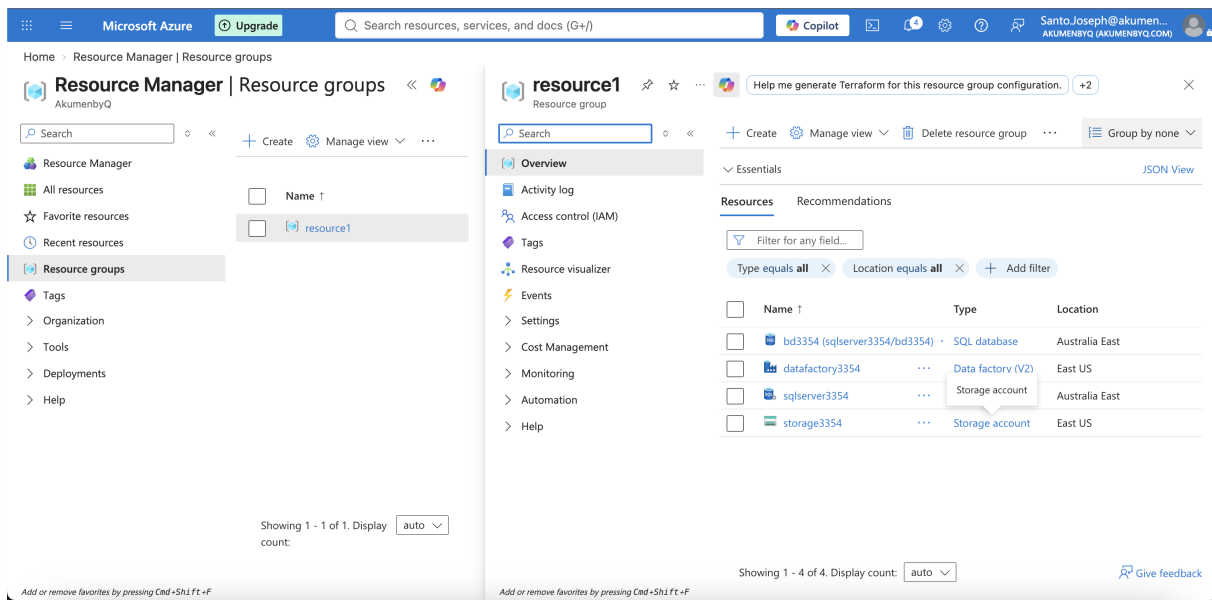
Step-by-step walkthrough: Azure SQL Database + Azure Data Factory

This guide documents the process of setting up an incremental loading pipeline using the watermark pattern. A watermark is a marker (usually the latest timestamp) that tracks how much data has already been processed, so that only new or changed rows are picked up on each pipeline run instead of reloading the entire source table every time.

Part 1 — Setting Up the Azure Environment

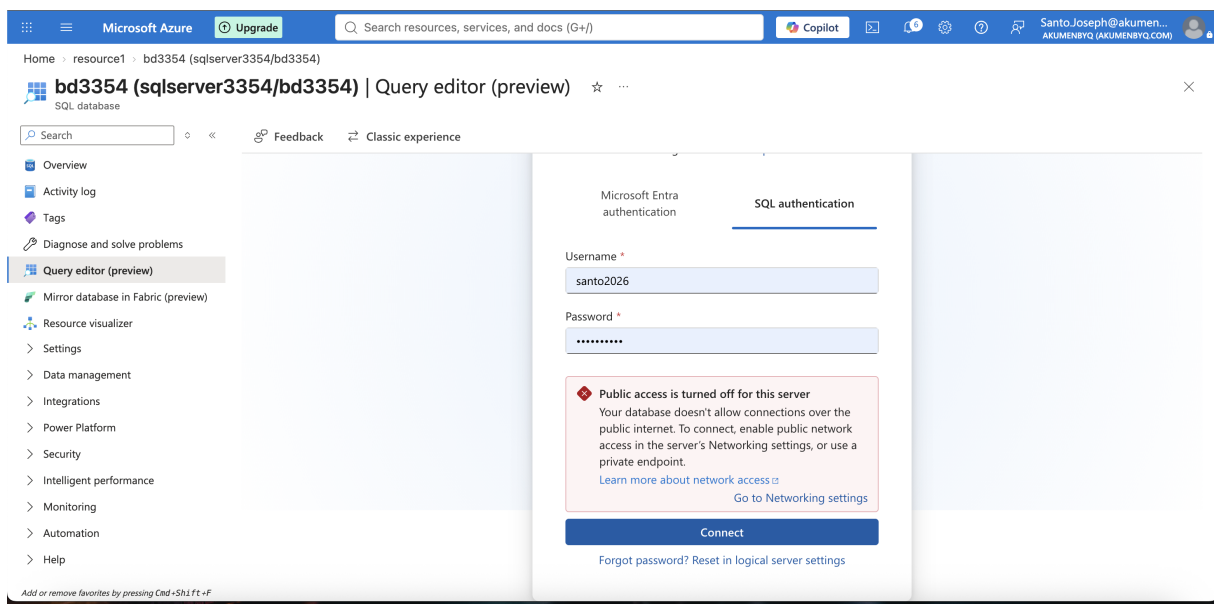
Step 1: Review the Resource Group

Open the resource group (resource1) containing the resources needed: a SQL Server / SQL Database (sqlserver3354 / bd3354), a Data Factory (datafactory3354), and a Storage account (storage3354).



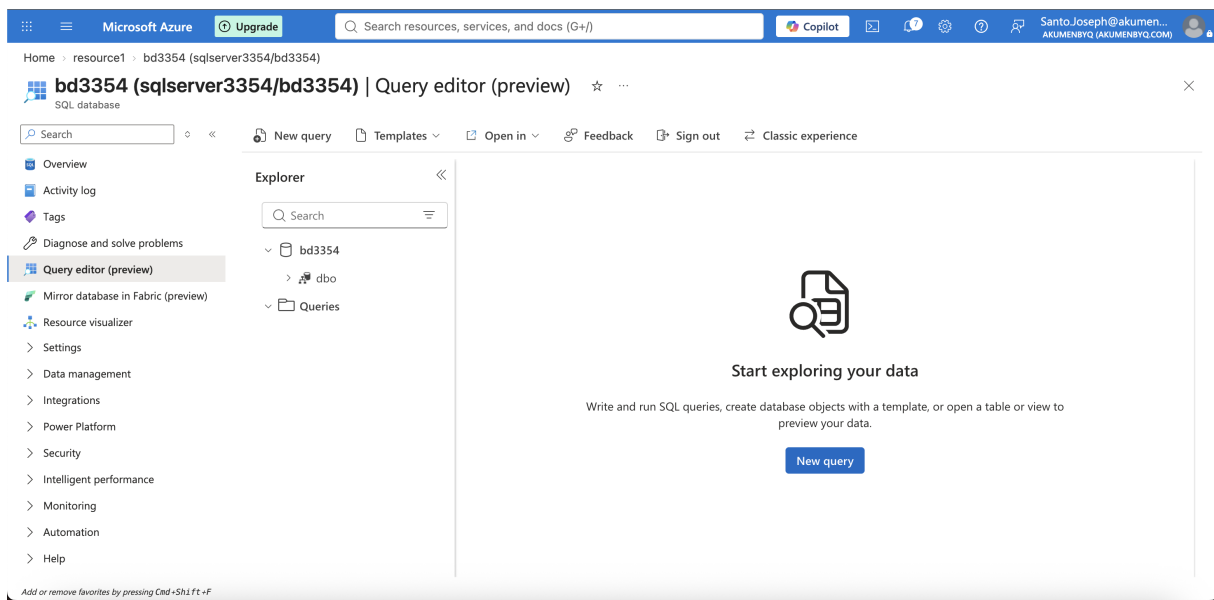
Step 2: Open the Query Editor and Sign In

Navigate to the SQL database (bd3354) and open Query editor (preview). Sign in using SQL authentication with a username and password. The first attempt shows that public access is turned off for the server.



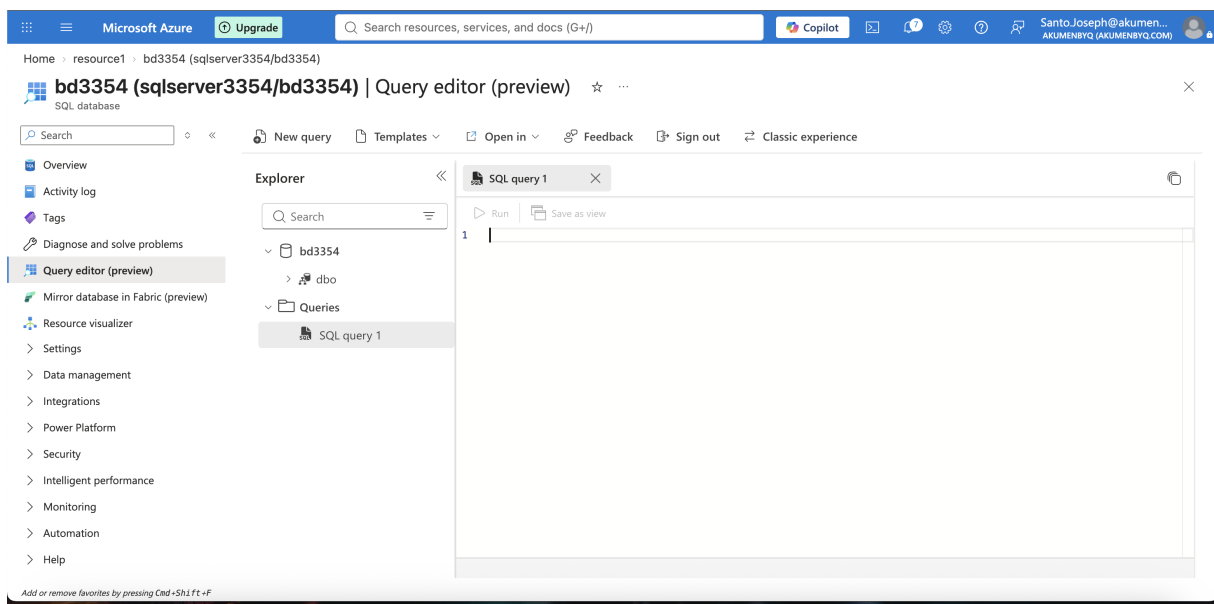
Step 3: Open a New Query

After enabling public network access on the server, the connection succeeds. The Query editor loads with the Explorer pane (dbo schema, Queries folder).



Step 4: Empty Query Window

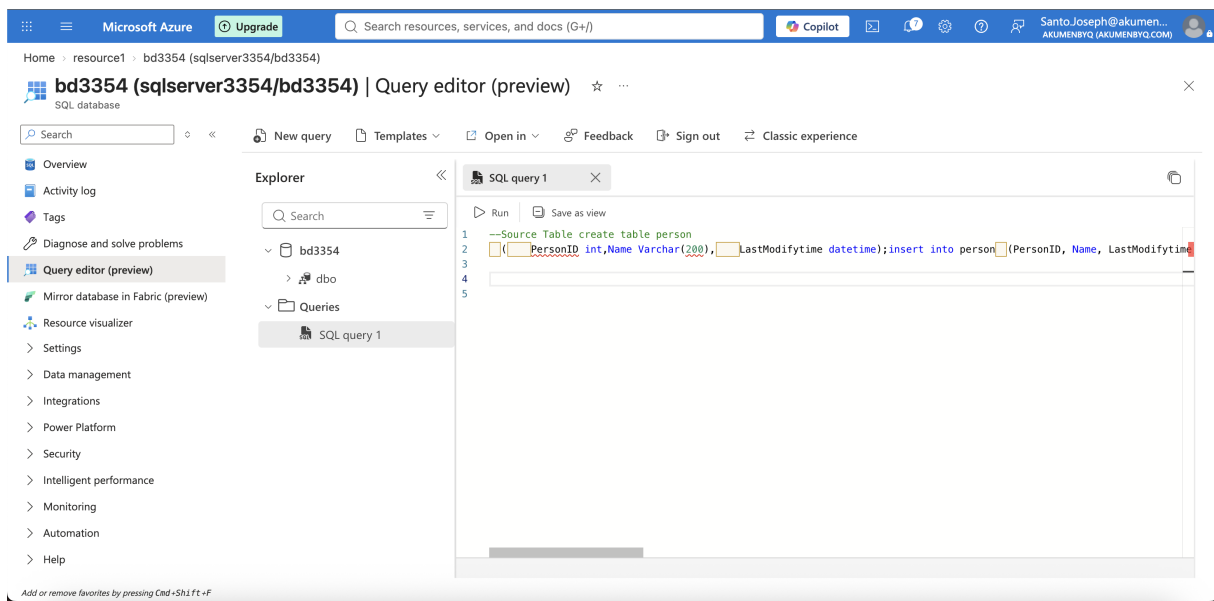
A new query tab, SQL query 1, is created and ready for input.



Part 2 — Preparing the Source Data

Step 5: Write the Table Creation Script

Write a script that creates a person table with three columns: PersonID (int), Name (varchar), and LastModifytime (datetime), followed by insert statements for five sample rows.



Step 6: Create the Table and Insert Sample Data

Running the script succeeds. The Results grid confirms 5 rows were inserted (Tiya, Miya, Ziya, Niya, Jiya), each with its own LastModifytime value ranging from 2019-07-01 to 2019-07-05.

Microsoft Azure | Upgrade | Search resources, services, and docs (G+)

Home > resource1 > bd3354 (sqlserver3354/bd3354)

bd3354 (sqlserver3354/bd3354) | Query editor (preview)

SQL database

Search

Overview
Activity log
Tags
Diagnose and solve problems
Query editor (preview)
Mirror database in Fabric (preview)
Resource visualizer
Settings
Data management
Integrations
Power Platform
Security
Intelligent performance
Monitoring
Automation
Help

Explorer

bd3354
dbo
Queries
SQL query 1

SQL query 1

```
--Incremental Loading using watermark
--Source Table
create table person
(
    PersonID int,
    Name Varchar(200),
    LastModifytime datetime
);
```

Messages Results

PersonID	Name	LastModifytime
1	Tiya	2019-07-01T00:55:00.0000000
2	Miya	2019-07-02T05:24:00.0000000
3	Ziya	2019-07-03T02:36:00.0000000
4	Niya	2019-07-04T03:20:00.0000000
5	Jiya	2019-07-05T08:05:00.0000000

Succeeded (0 sec 329 ms) Columns: 3 Rows: 5

Step 7: Create the Watermark Table

Create a second table, watermarktable, with columns TableName (varchar) and Watermarkvalue (datetime), then insert one row: TableName = 'person', Watermarkvalue = 2000-01-01. This old date is the starting point so the first pipeline run captures all existing rows.

Microsoft Azure | Upgrade | Search resources, services, and docs (G+)

Home > resource1 > bd3354 (sqlserver3354/bd3354)

bd3354 (sqlserver3354/bd3354) | Query editor (preview)

SQL database

Search

Overview
Activity log
Tags
Diagnose and solve problems
Query editor (preview)
Mirror database in Fabric (preview)
Resource visualizer
Settings
Data management
Integrations
Power Platform
Security
Intelligent performance
Monitoring
Automation
Help

Explorer

bd3354
dbo
Queries
SQL query 1

SQL query 1

```
--Watermark Table
create table watermarktable
(
    TableName varchar(200),
    Watermarkvalue datetime
);
insert into watermarktable
(
    TableName,
    Watermarkvalue
)
values
(
    'person',
    '2000-01-01T00:00:00.0000000'
);
```

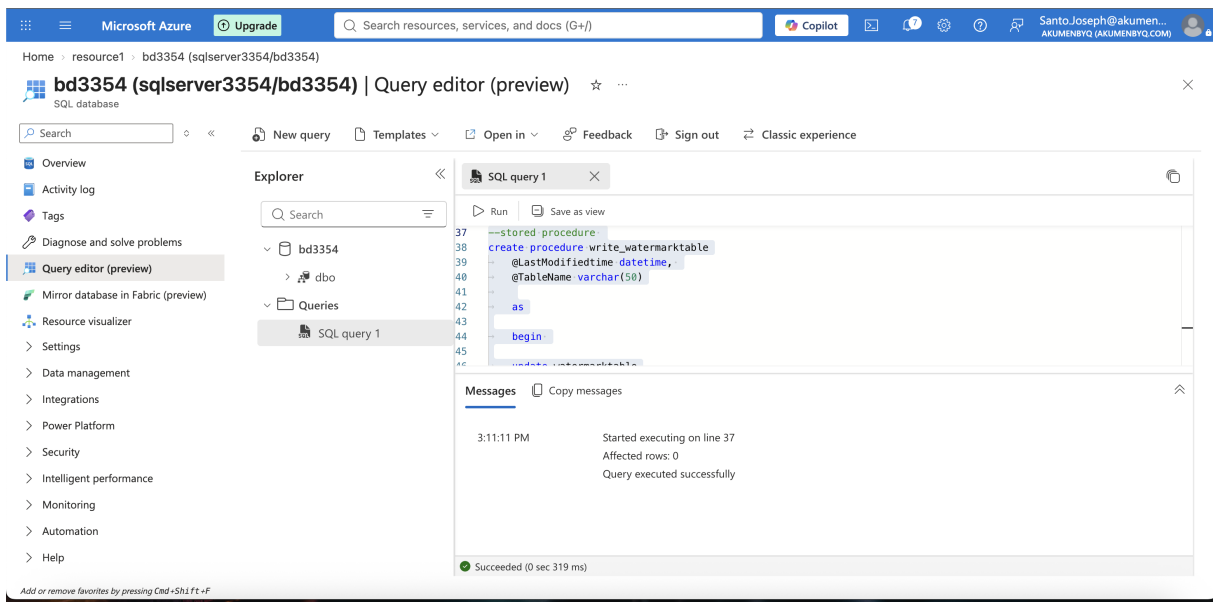
Messages Results

TableName	Watermarkvalue
person	2000-01-01T00:00:00.0000000

Succeeded (0 sec 324 ms) Columns: 2 Rows: 1

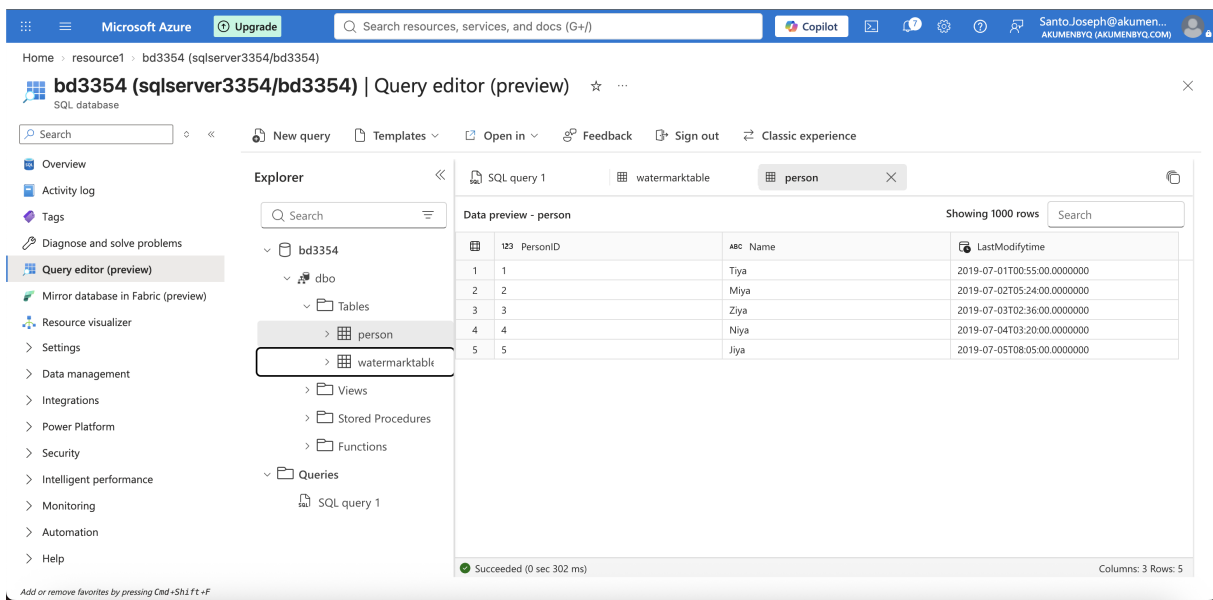
Step 8: Create a Stored Procedure to Update the Watermark

Create a stored procedure named write_watermarktable that accepts a @LastModifiedtime and @TableName parameter and updates watermarktable after each successful load. The script runs successfully.



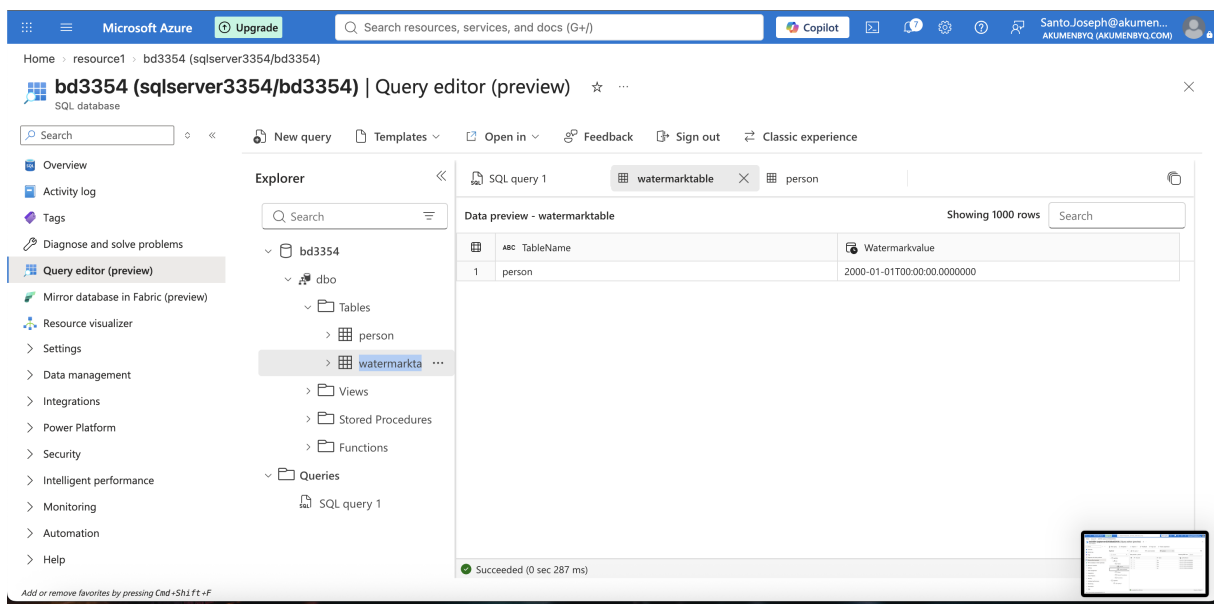
Step 9: Browse the person Table

In the Explorer pane, expand Tables and preview the person table's data to confirm the 5 inserted rows are present.



Step 10: Browse the watermarktable

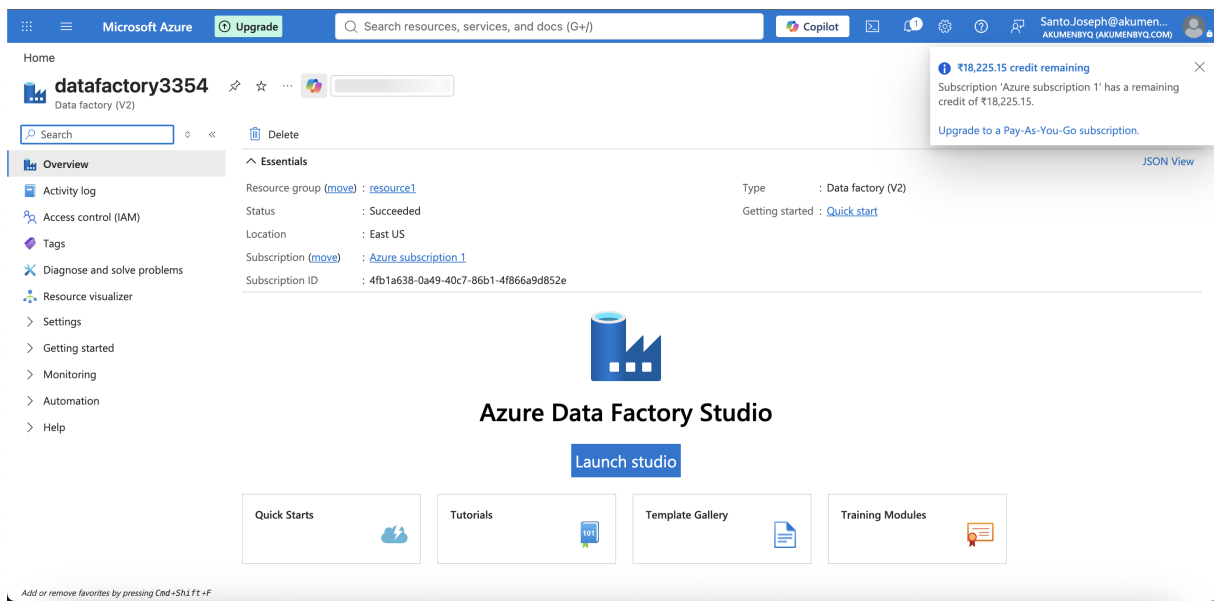
Preview watermarktable to confirm the single baseline row: person, 2000-01-01T00:00:00.



Part 3 — Building the Pipeline in Data Factory

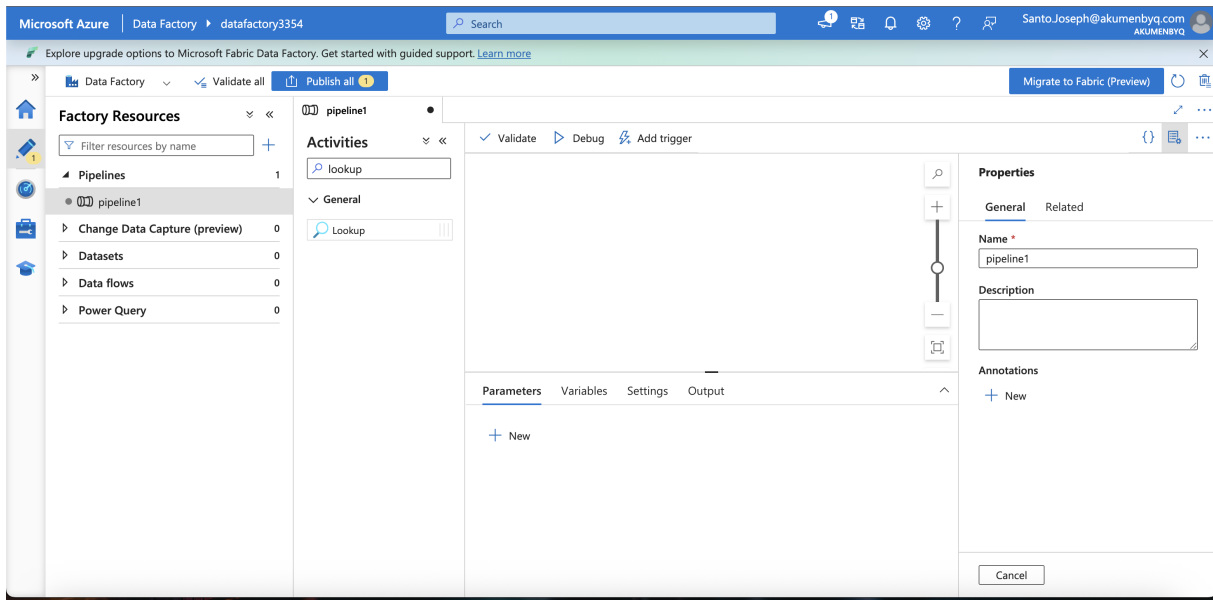
Step 11: Open Data Factory Studio

From the resource group, open datafactory3354 and click Launch studio to enter the Azure Data Factory authoring environment.



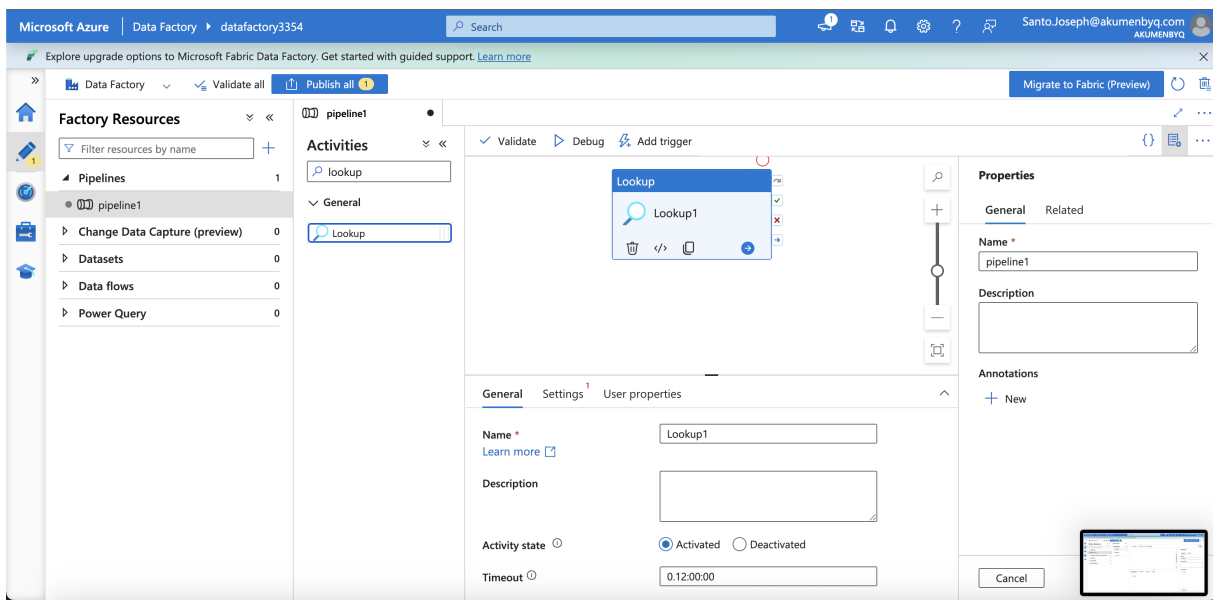
Step 12: Create a New Pipeline

Under Factory Resources, Pipelines, create a new pipeline named pipeline1. This opens the pipeline canvas with an Activities panel on the left.



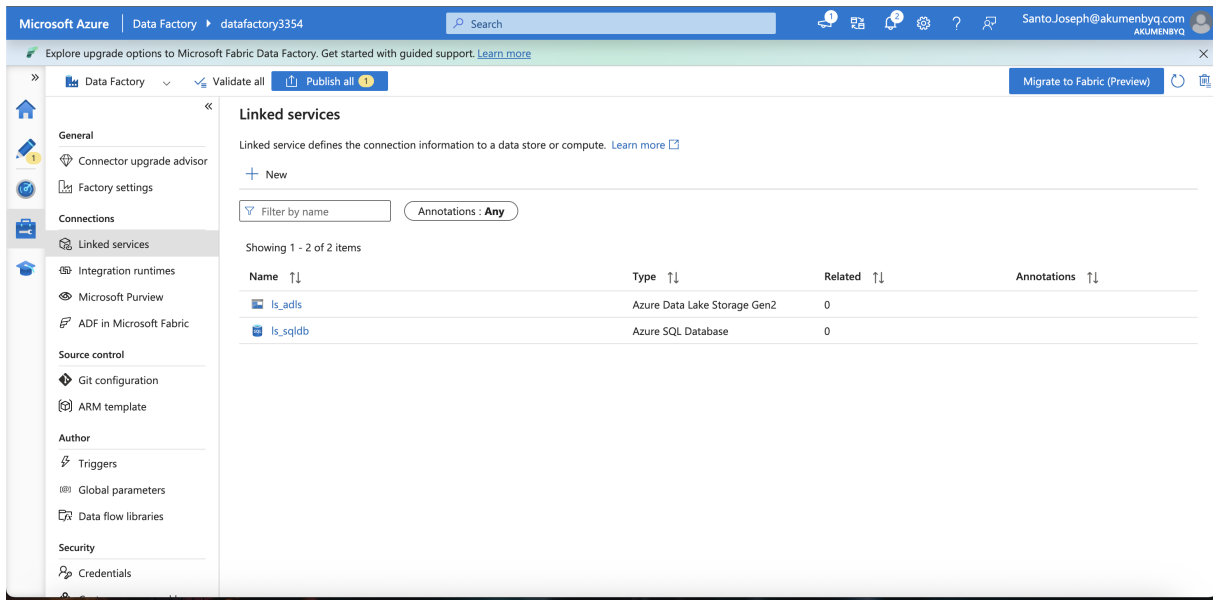
Step 13: Add the First Lookup Activity

Search for Lookup in the Activities panel and drag it onto the canvas. Name it Lookup1 — this activity will read the current watermark value from watermarktable.



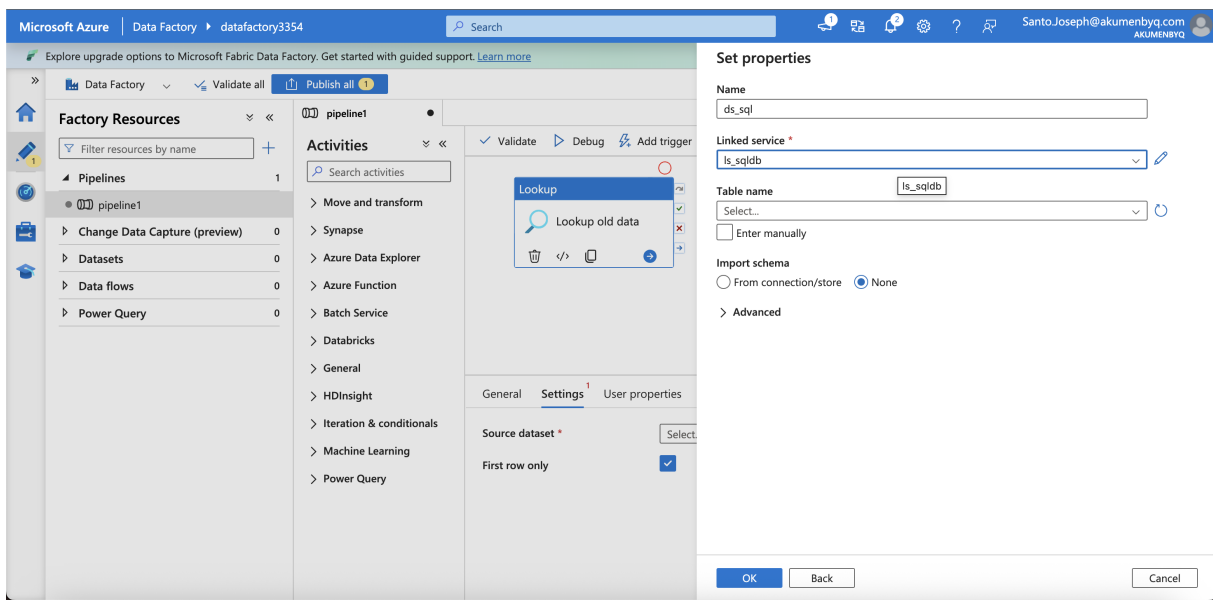
Step 14: Review Linked Services

Under Manage, Linked services, confirm the two linked services already created: ls_adls (Azure Data Lake Storage Gen2) and ls_sqldb (Azure SQL Database).



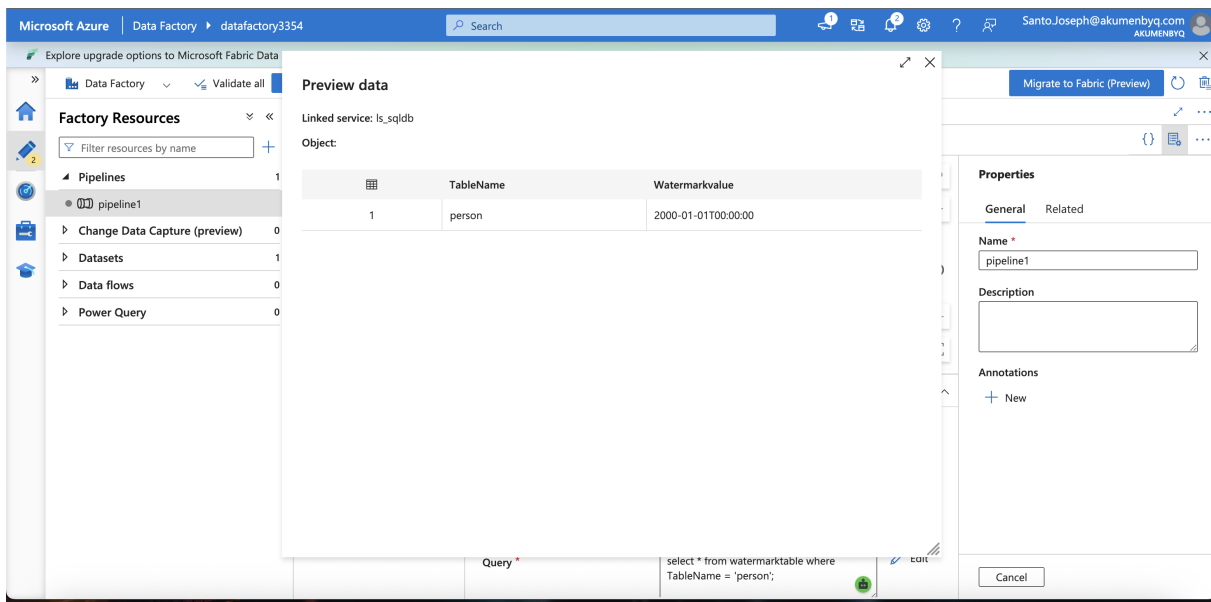
Step 15: Create the Dataset for the Lookup

Configure the Lookup activity's Settings tab by creating a new dataset named ds_sql, pointing to the linked service ls_sqldb. Schema import is set to None since the query defines the output.



Step 16: Preview the Old Watermark

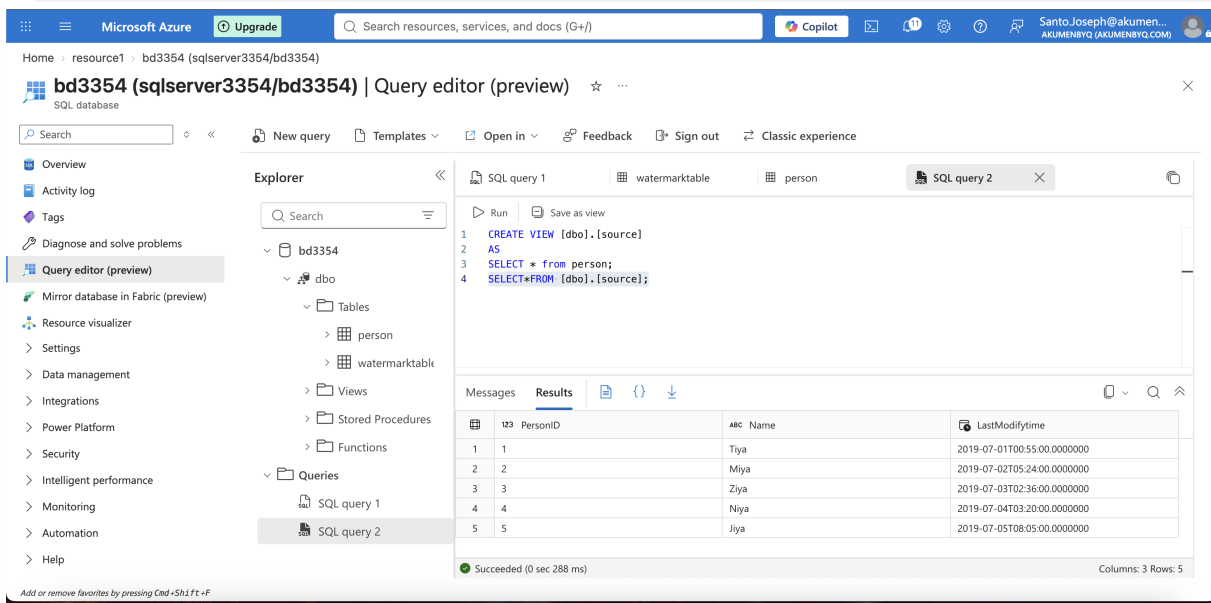
Using Preview data confirms the Lookup activity correctly retrieves the baseline row from watermarktable: TableName = person, Watermarkvalue = 2000-01-01T00:00:00.



Step 17: Create a View of the Source Table

Back in the Query editor, create a SQL view over the source table so it can be referenced cleanly from the pipeline:

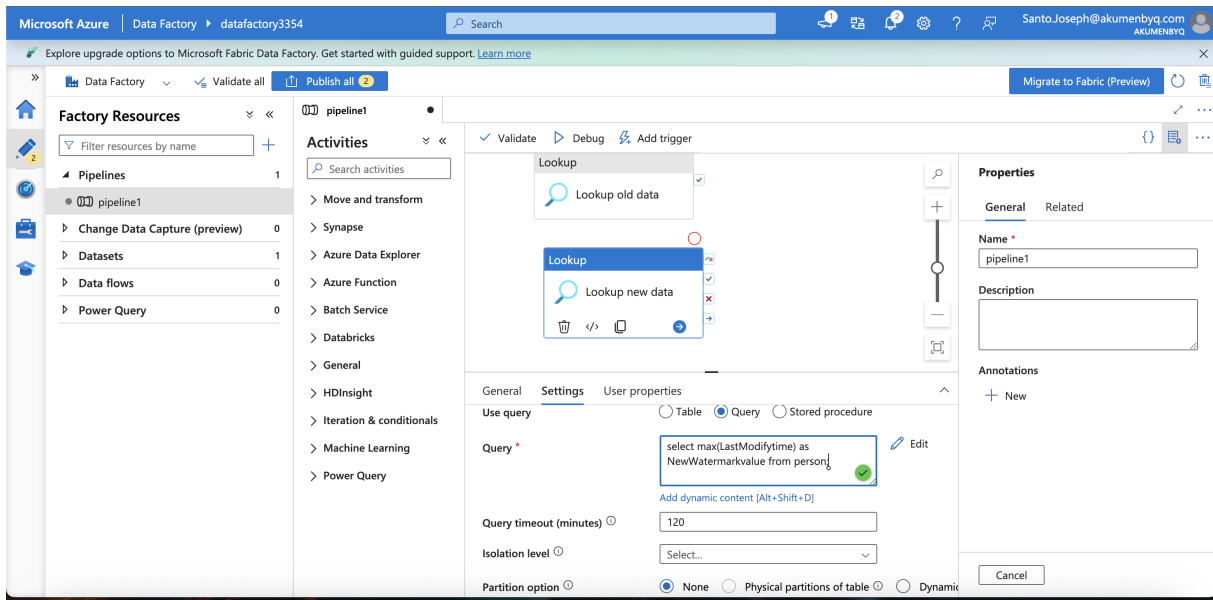
```
CREATE VIEW [dbo].[source] AS SELECT * from person; SELECT*FROM [dbo].[source];
```



Step 18: Add a Second Lookup Activity

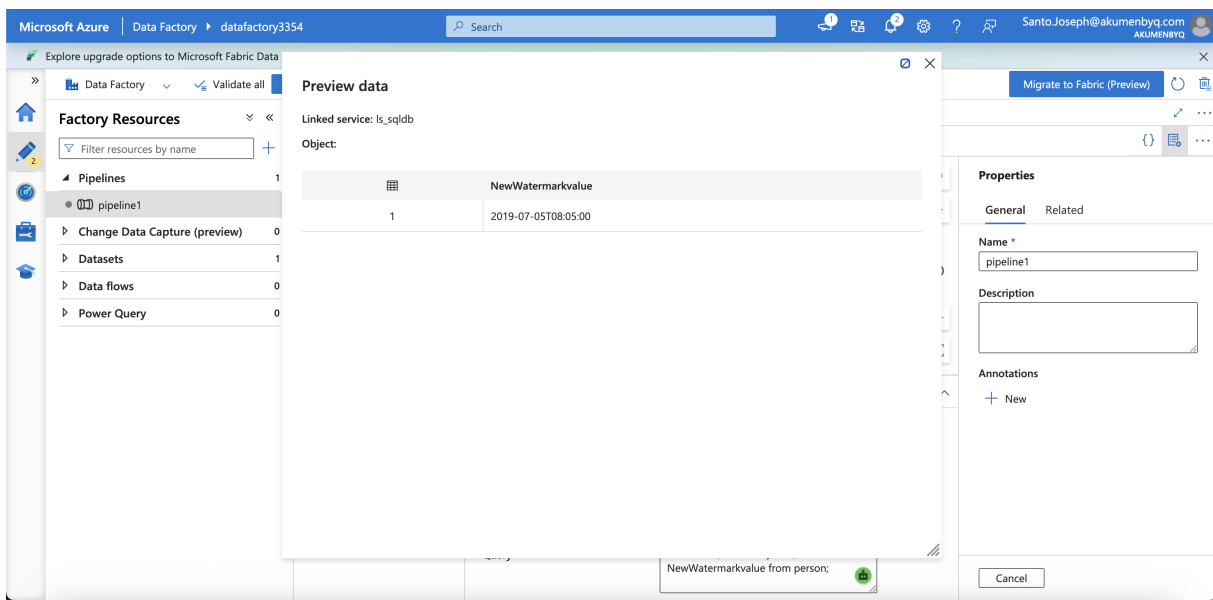
Add another Lookup activity, named Lookup new data. In its Settings tab, set the query type to Query and enter a custom SQL statement that finds the most recent LastModifytime in the source table:

```
select max(LastModifytime) as NewWatermarkvalue from person
```



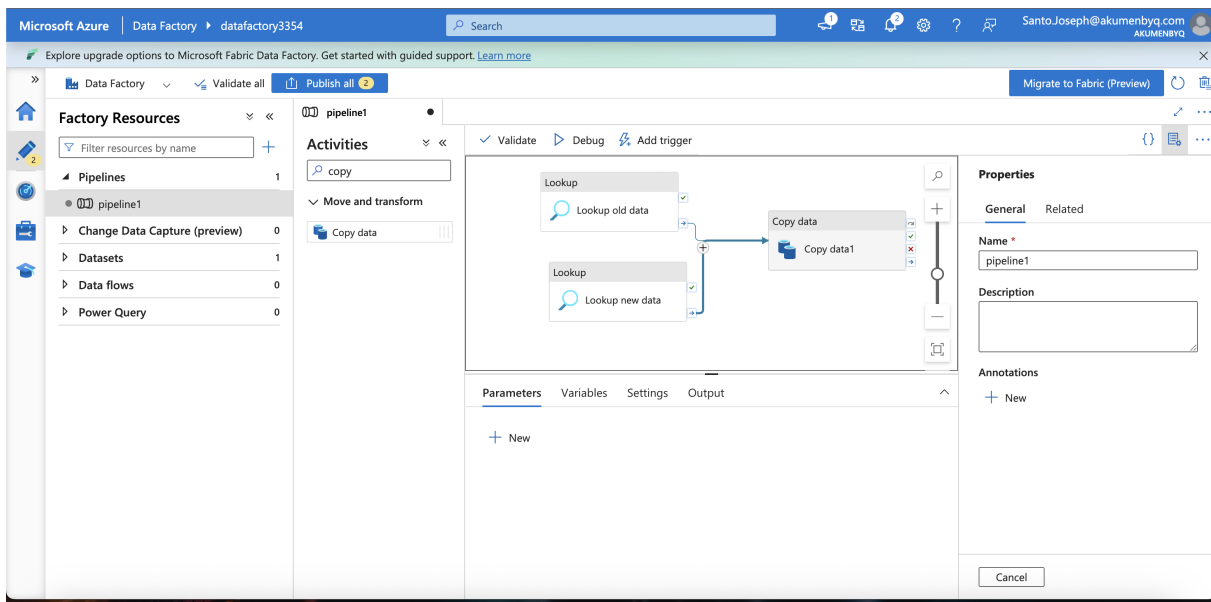
Step 19: Preview the New Watermark

Previewing this Lookup activity's output confirms it returns NewWatermarkvalue = 2019-07-05T08:05:00, the latest LastModifytime among the 5 rows in the person table.



Step 20: Add the Copy Data Activity

Add a Copy data activity to the canvas and connect the On Success output of both Lookup activities into it. The Copy activity will use the old and new watermark values to select only the rows that changed since the last run, and copy them to the destination.

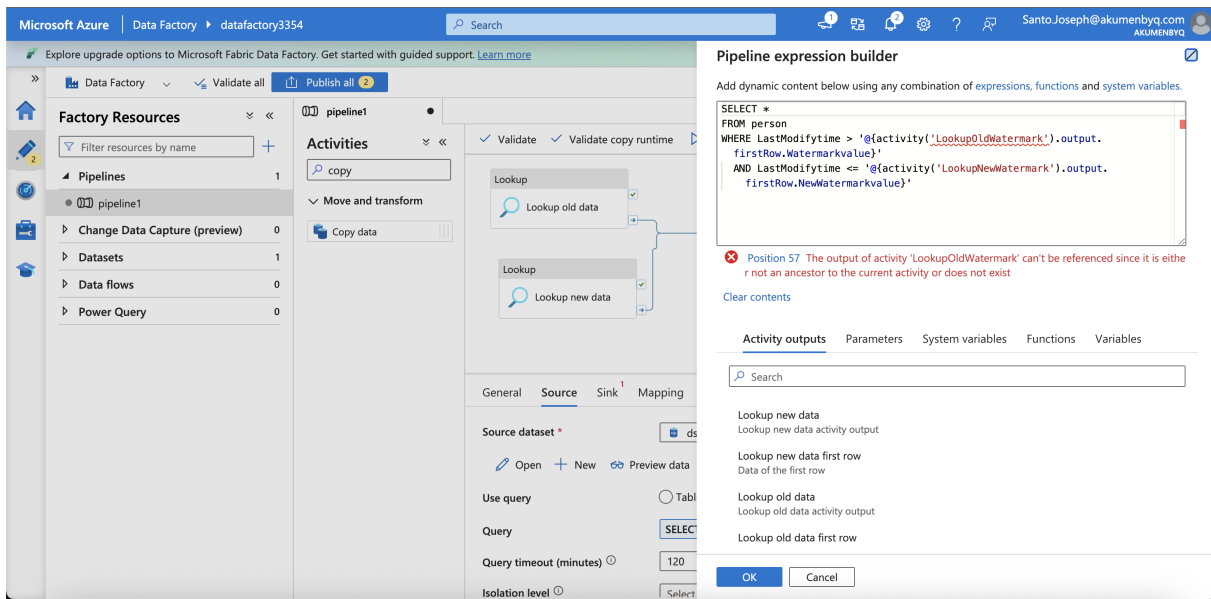


Part 4 — Configuring the Copy and Update Steps

Step 21: Build the Source Query with Dynamic Content

In the Copy data activity's Source tab, open the Pipeline expression builder to write a query that filters person rows using the two watermark values. The first attempt uses placeholder activity names (LookupOldWatermark / LookupNewWatermark) that do not match the actual activity names, producing a validation error: "The output of activity 'LookupOldWatermark' can't be referenced since it is either not an ancestor to the current activity or does not exist."

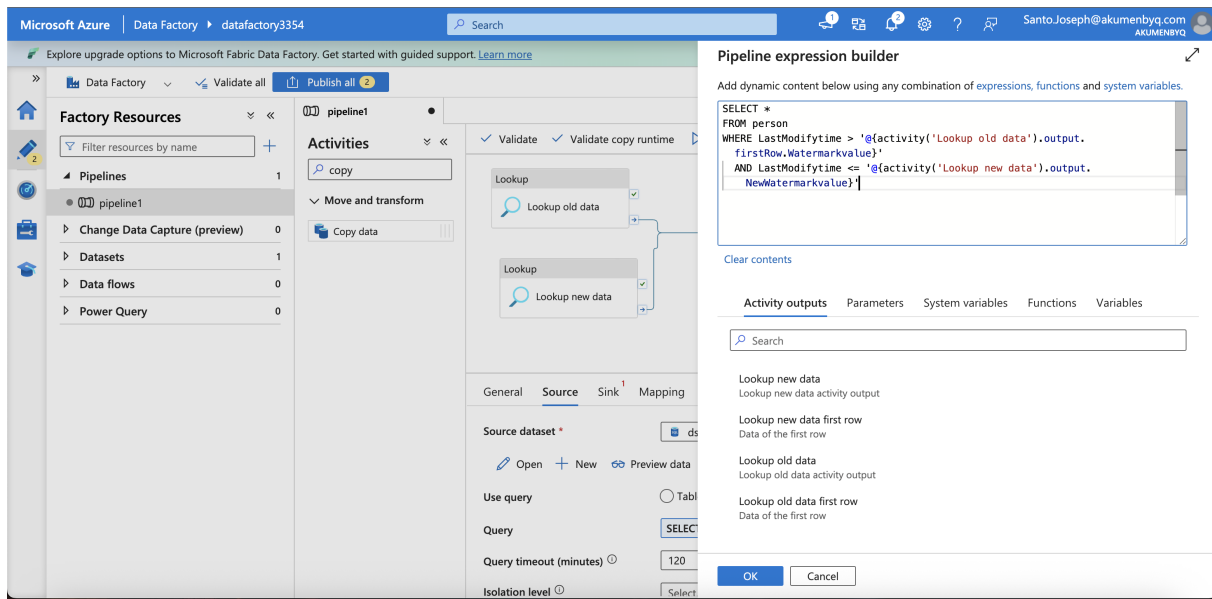
```
SELECT * FROM person WHERE LastModifytime >
'@{activity('LookupOldWatermark').output.firstRow.Watermarkvalue}' AND LastModifytime <=
'@{activity('LookupNewWatermark').output.firstRow.NewWatermarkvalue}'
```



Step 22: Correct the Expression

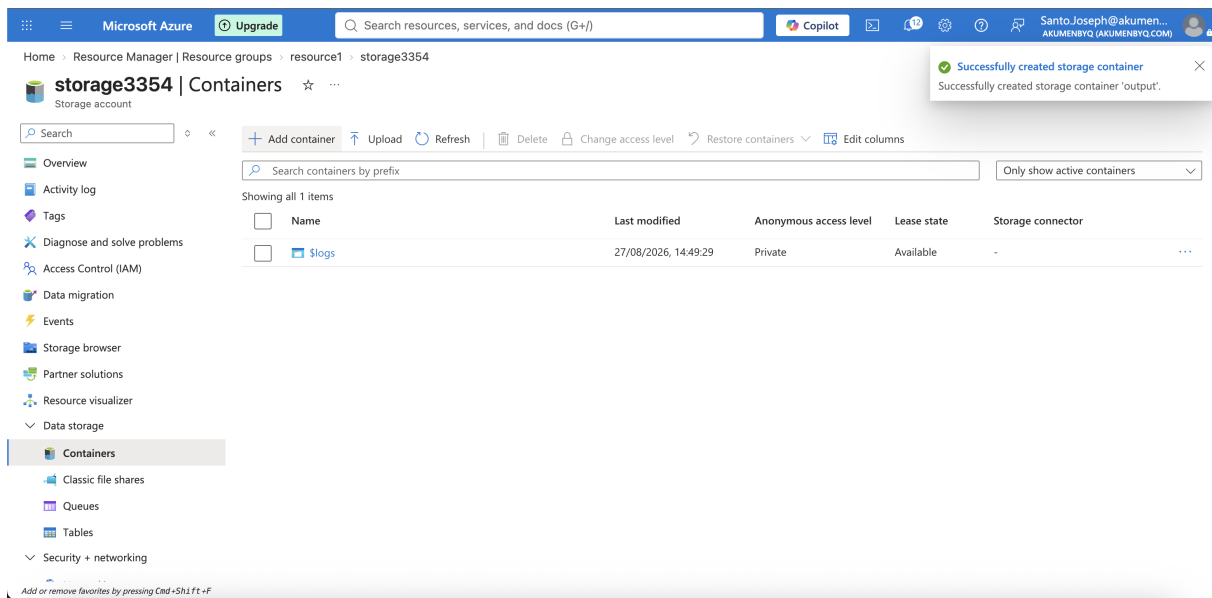
The expression is corrected to reference the actual activity names, 'Lookup old data' and 'Lookup new data', matching what is on the canvas. The error clears and the query is accepted.

```
SELECT * FROM person WHERE LastModifytime > '@{activity('Lookup old data').output.firstRow.Watermarkvalue}' AND LastModifytime <= '@{activity('Lookup new data').output.NewWatermarkvalue}'
```



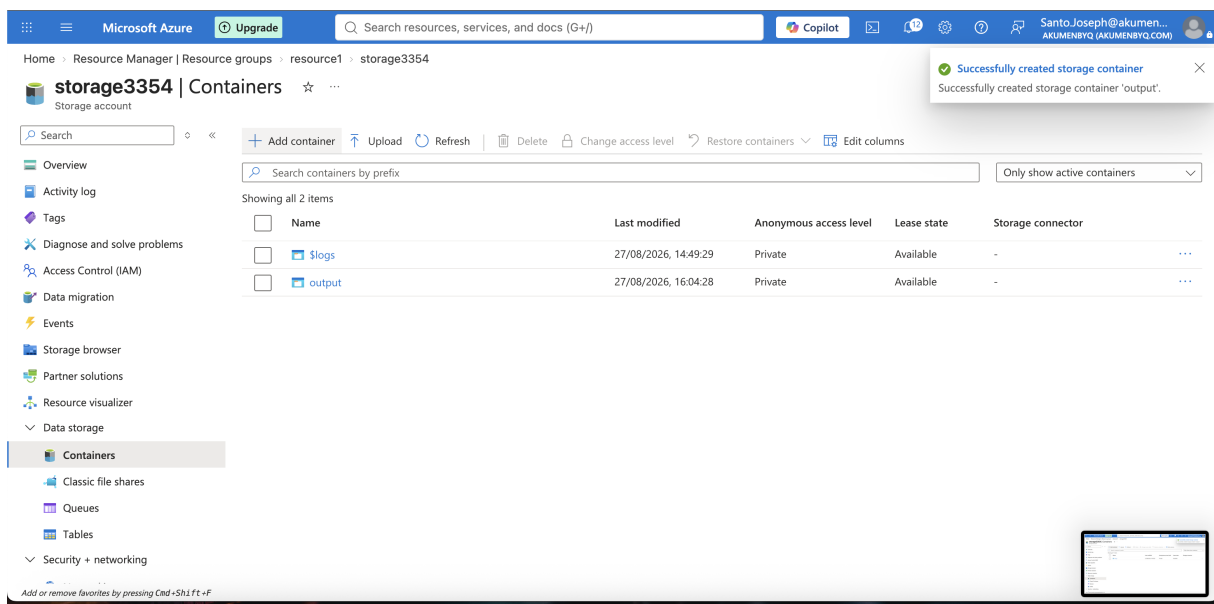
Step 23: Create an Output Container in Storage

In the storage account (storage3354), go to Containers and add a new container named output. A confirmation message appears: "Successfully created storage container 'output'."



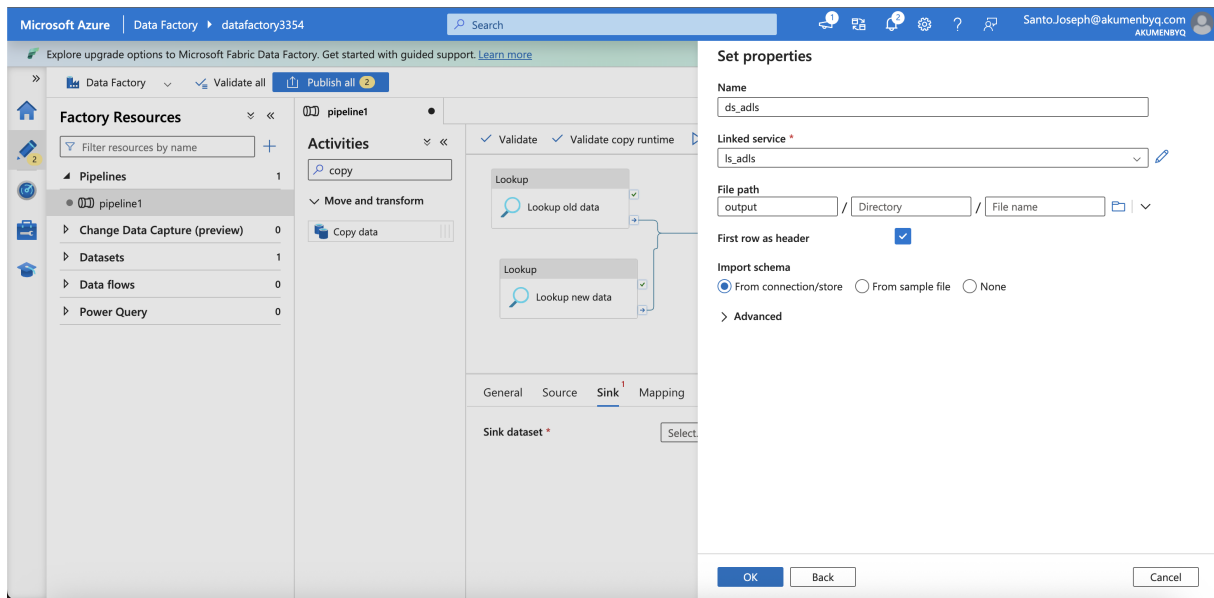
Step 24: Confirm the Container Exists

The Containers list now shows two items: the default \$logs container and the new output container, confirming it was created successfully.



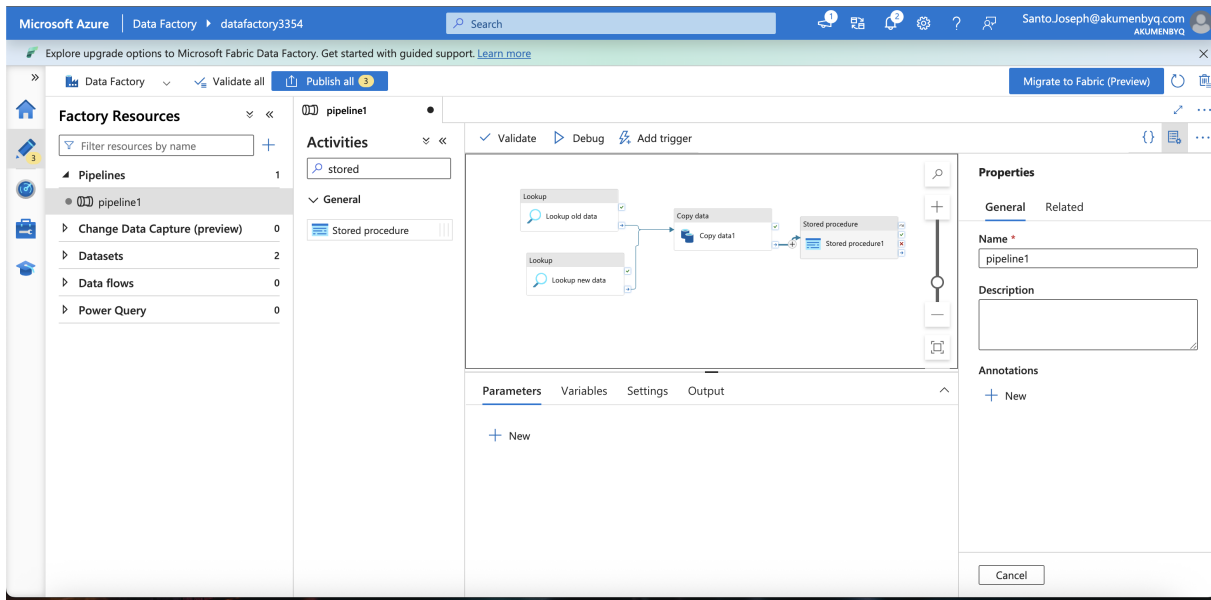
Step 25: Create the Sink Dataset

Back in the Copy data activity's Sink tab, create a new dataset named `ds_adls`, linked to `ls_adls`, pointing to the output container as the file path. First row as header is checked, and schema import is set to From connection/store.



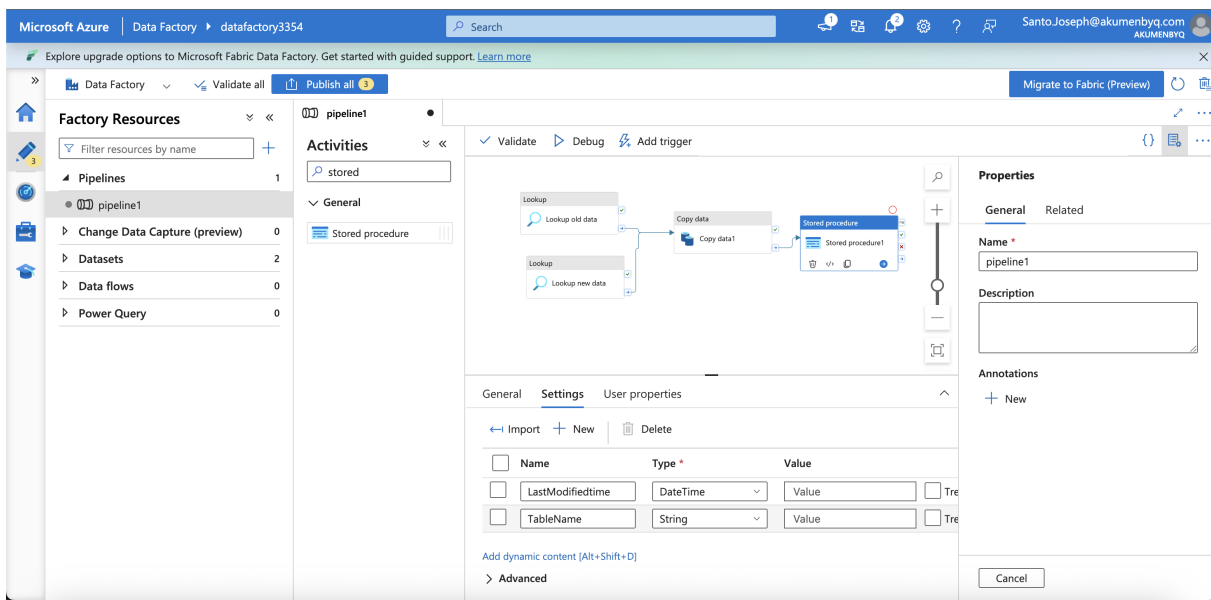
Step 26: Add a Stored Procedure Activity

Add a Stored procedure activity to the pipeline canvas, connected after the Copy data activity's On Success output. This activity will call `write_watermarktable` to update watermarktable with the new watermark value once the copy completes.



Step 27: Configure the Stored Procedure Parameters

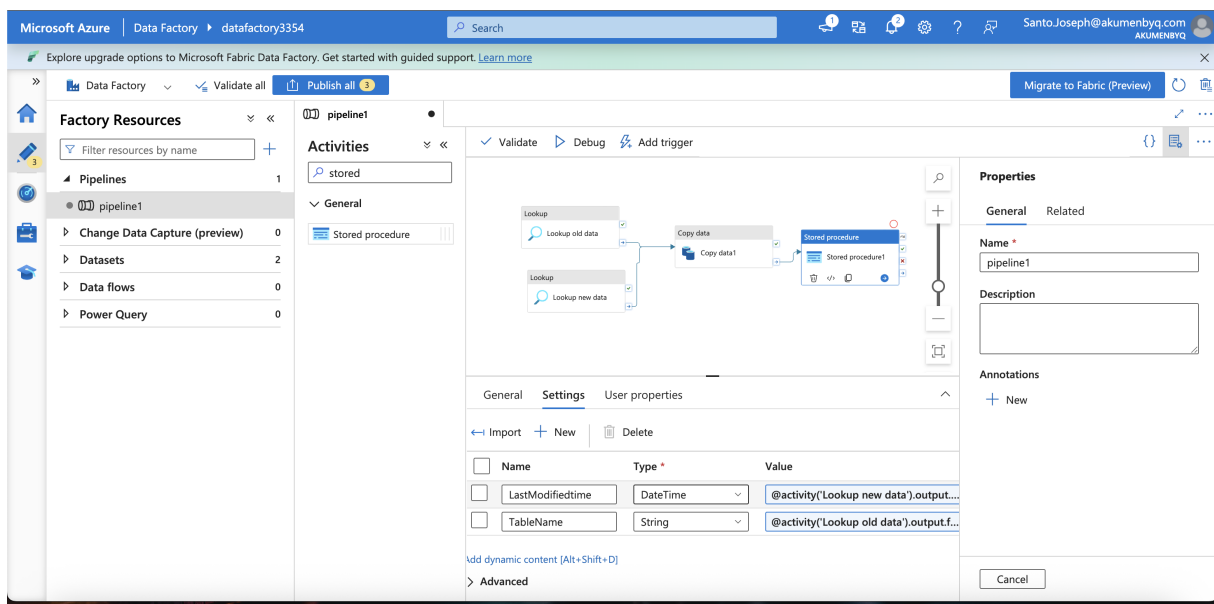
In the Stored procedure activity's Settings tab, import the stored procedure's parameters: LastModifiedtime (DateTime) and TableName (String). At this point both Value fields are still empty.



Step 28: Bind the Parameters to Pipeline Outputs

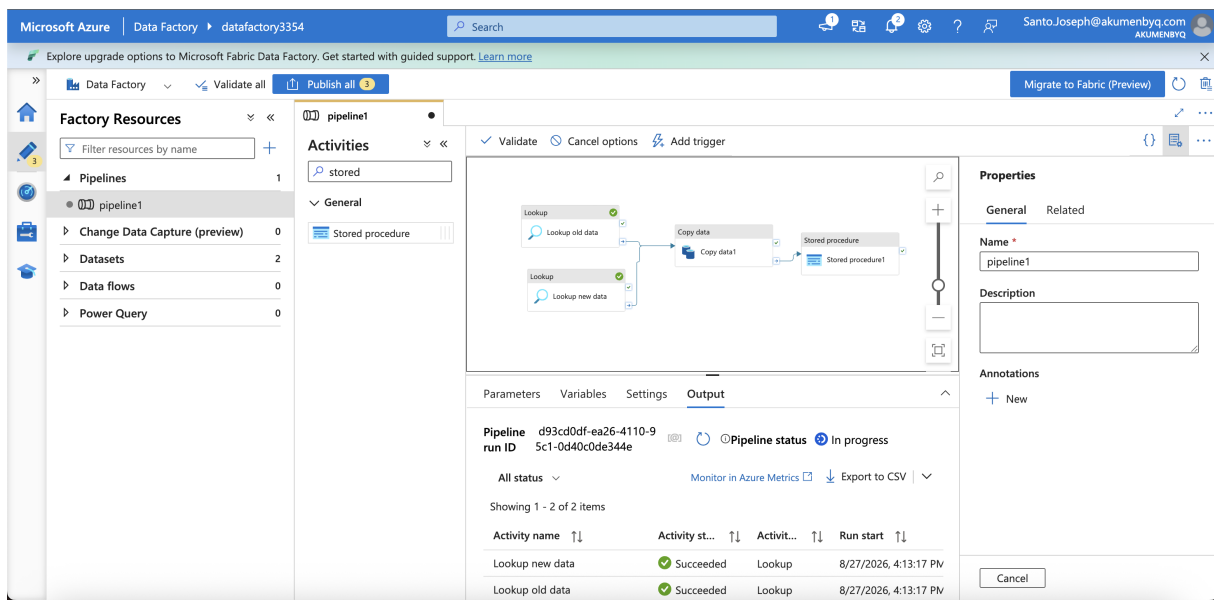
Set the parameter values using dynamic content so the stored procedure receives live values from the Lookup activities: LastModifiedtime is bound to the new watermark from Lookup new data, and TableName is bound to the table name from Lookup old data.

```
LastModifiedtime = @activity('Lookup new data').output.firstRow.NewWatermarkvalue
TableName = @activity('Lookup old data').output.firstRow.TableName
```



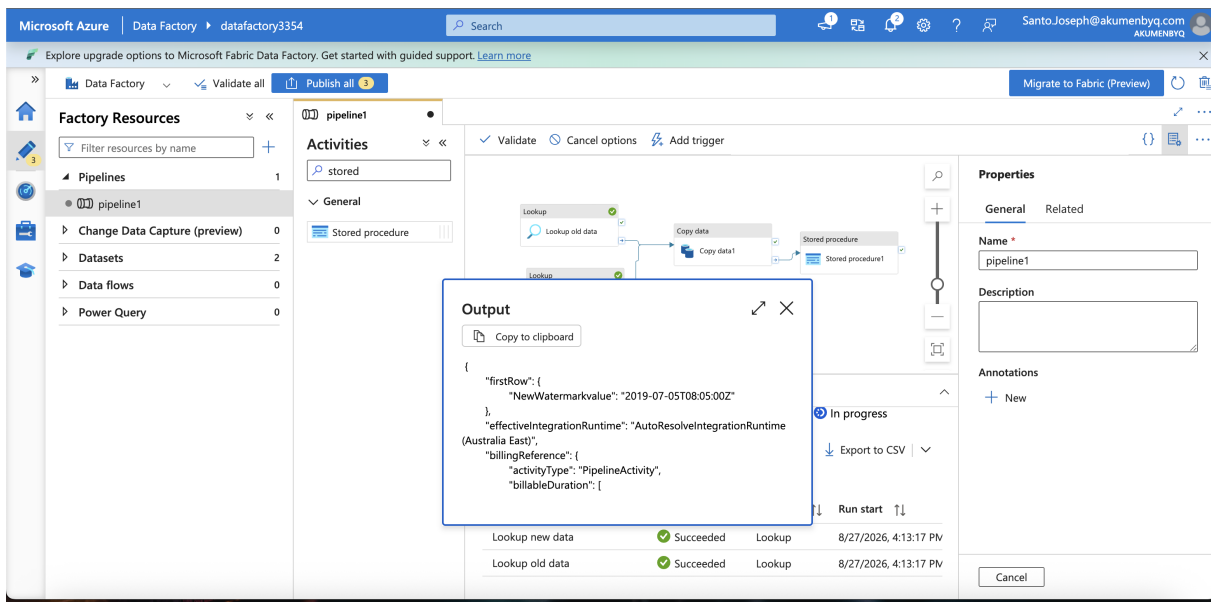
Step 29: Debug the Pipeline

Click Debug to run the pipeline. The Output tab shows the run in progress, with both Lookup activities (Lookup new data and Lookup old data) completing successfully.



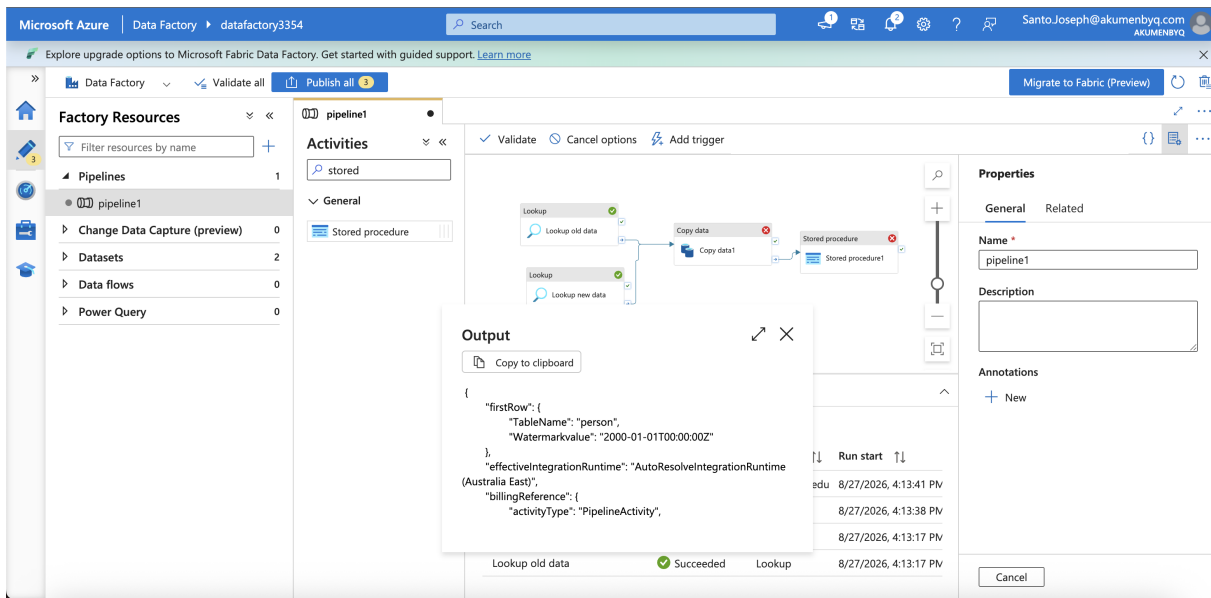
Step 30: Inspect the Lookup new data Output

Expanding the output of Lookup new data confirms it correctly returns NewWatermarkvalue = 2019-07-05T08:05:00Z, the latest LastModifytime from the source table.



Step 31: Inspect the Lookup old data Output

Expanding the output of Lookup old data confirms it correctly returns TableName = person, Watermarkvalue = 2000-01-01T00:00:00Z, the baseline watermark. The Copy data and Stored procedure activities show error icons in this run — the next step is to open those activities and resolve the failure before re-running the debug.



How the Pattern Works, End to End

1. The watermark table remembers the last timestamp that was successfully loaded.
2. On each run, Lookup old data reads that stored watermark value.
3. Lookup new data checks the source table for its current maximum timestamp.
4. Copy data uses a dynamic query, built from both Lookup activities' outputs, to transfer only the rows with a timestamp between the old and new watermark — i.e. just the new or changed records — into the output container in storage.

5. A Stored procedure activity then updates the watermark table with the new value (using dynamic content bound to the Lookup outputs), so the next run starts exactly where this one left off.